a project on

Real-Time Mobile Phone Usage Detection and Student Identification Using Machine Learning

Submitted by

Sriram Revanth Sai - AP23110011112

Bachelor of Technology
In
Computer Science and Engineering
School of Engineering and Sciences

Under the guidance of

Mr. Piyush Kumar R

[November, 2024]

SRM University, AP, Neerukonda, Mangalagiri, Guntur.

Andhra Pradesh – 522 240

Department of Computer Science and Engineering

SRM University, AP

CERTIFICATE

*This is to certify that the Project report entitled "**Real-Time Mobile Phone Usage Detection and Student Identification Using Machine Learning**" is being submitted by **Sriram Revanth Sai(AP23110011112) ,B.Rohith (AP23110010089),Madhan Reddy(AP23110011124) ,Chaitanya(AP23110010029) , G. Nitesh Kumar (AP23110010361)** students of Department of Computer Science and Engineering, SRM University, AP, in partial fulfillment of the requirement for* ***Machine Learning Project*** *for III-B. Tech (CSE), Semester V. carried out by them during the academic year 2024-2025.*

Signature of the Lab In charge

Mr. Piyush Kumar R

Assistant Professor,Department of CSE

TABLE OF CONTENTS

Name of the content **Page. No**

1. Acknowledgment
2. Abstract
3. Introduction
4. Motivation
5. Objectives
6. Methodology
7. System Architecture
8. Implementation
9. Results & Analysis
10. Limitations
11. Future Scope
12. Conclusion

1. Acknowledgment

We express our heartfelt gratitude to Mr. Piyush Kumar R, Department of Computer Science and Engineering, SRM University AP, for his consistent support, mentorship, and encouragement throughout the development of this project. His insights on machine learning techniques, real-time systems, and project design have been invaluable to us.

We would also like to thank the faculty members of the Department of CSE for providing necessary resources, such as lab infrastructure, tools, references, and a conducive academic environment.

Finally, we acknowledge the sincere efforts and collaboration of all team members. Each member contributed equally to the successful completion of this project.

2. Abstract

Mobile phones have become a major source of distraction in classrooms, often affecting student attentiveness, academic performance, and overall discipline. Manual monitoring is inefficient, error-prone, and requires continuous human supervision. To address this challenge, this project introduces a real-time machine learning-based monitoring system capable of detecting mobile phone usage and identifying the student involved.

The system uses YOLOv8, a state-of-the-art deep learning–based object detection model, to detect mobile phones in live video. For identifying students, the project uses a face recognition system, trained using student-specific datasets. The face embeddings are generated with the *face_recognition* library, which maps face features into 128-dimensional vectors.

The system integrates both models to trigger real-time alerts whenever a student uses a phone during class. The alert includes a captured snapshot and is automatically sent to the instructor's email. The project demonstrates strong alignment with machine learning concepts such as supervised learning, dataset preparation, feature extraction, CNN-based detection models, model evaluation, and real-time inference.

3. Introduction

Machine Learning (ML) has transformed the way intelligent systems perceive and interact with the world. From healthcare diagnostics to autonomous driving, ML's ability to analyze visual data has resulted in rapid adoption across industries.

In educational environments, maintaining student attention and minimizing distractions are key challenges. The increasing misuse of smartphones during lectures not only disturbs the learning environment but also makes it difficult for instructors to continuously monitor student behavior.

Traditional methods rely heavily on human observation. They suffer from several limitations:

- Not continuous
- Error-prone
- Easily bypassed
- Physically demanding

To overcome these issues, this project builds a **fully automated system** capable of:

- Detecting mobile phone usage
- Identifying which student used the phone
- Sending immediate alerts

The solution leverages **computer vision**, **deep learning**, and **face recognition**, making it a practical real-time application of Machine Learning.

4. Motivation

The primary motivation behind this project is to minimize classroom distractions and introduce a technology-driven monitoring solution. Mobile phones are helpful outside the classroom but are often misused during lectures.

The project aims to:

- Support teachers by reducing manual monitoring workload
- Improve classroom discipline
- Provide evidence-based alerts
- Explore real-time ML applications
- Demonstrate the effectiveness of deep learning in real-world environments

5. Objectives

The primary objectives of this Machine Learning project are as follows:

1. Detect Mobile Phone Usage in Real-Time

To design and implement a highly accurate mobile phone detection system using the YOLOv8 object detection model. The system must be capable of identifying the presence of a mobile phone in each video frame with high confidence, even in varying lighting conditions and classroom environments.

2. Identify the Student Using the Phone

To build a reliable face recognition module that extracts 128-dimensional embeddings from student images and matches them with real-time webcam input. This helps determine which student is using the mobile phone at any given moment.

3. Provide Real-Time Alerts to the Instructor

To automate the alert mechanism by sending an email notification containing:

- Student name
- Timestamp
- Evidence snapshot

  This helps the instructor take immediate action without continuously monitoring the system.

4. Enable Continuous and Automated Classroom Monitoring

To develop a system that operates continuously using the laptop webcam without manual intervention. The monitoring loop runs throughout the lecture session and detects prohibited behavior instantly.

5. Build a Complete End-to-End Machine Learning Pipeline

To demonstrate the practical application of ML by completing all essential components, including:

- Dataset creation (student face images)
- Preprocessing and face encoding
- Training and storing embeddings
- Real-time detection and inference
- Performance evaluation and testing
- Integration of detection, recognition, and alerting into one system

6. Improve Classroom Discipline and Reduce Teacher Workload

To create a smart monitoring assistant that reduces the burden on instructors by automating the detection of classroom distractions and improving overall discipline.

6. Methodology

The proposed system follows a structured and systematic Machine Learning pipeline to ensure accurate detection and identification. The complete methodology is divided into six major stages as described below.

Step 1: Dataset Collection

Student face images were collected using the laptop webcam in controlled lighting conditions.
Each student contributed a minimum of 20 images with slight variations in:

- Angle (left, right, front)
- Facial expressions
- Distance from the camera

The images were stored in separate folders for each student, forming the training dataset.
This dataset acts as the foundation for the face recognition model.

Step 2: Image Preprocessing

Before training, each collected image undergoes essential preprocessing steps:

- Color conversion: Images are converted to RGB format since face_recognition works best with RGB channels.
- Face cropping: Haar Cascade classifiers are used to detect and crop the face region precisely.
- Resizing and normalization: Cropped faces are resized to a fixed size, and pixel values are normalized to reduce variability.

These preprocessing steps significantly improve the accuracy of face encoding and ensure consistency.

Step 3: Face Encoding and Training

In this stage, the processed face images are fed to the face_recognition library, which internally uses a deep CNN model to convert each face into a 128-dimensional embedding vector.

- Each embedding represents unique facial features of the student.
- All embeddings are stored in encodings.npy, and corresponding names are stored in names.npy.

These files act as the trained face recognition model, enabling quick comparison during real-time inference.

Step 4: Real-Time Mobile Phone Detection Using YOLOv8

To detect mobile phone usage, the YOLOv8n object detection model (Ultralytics) is integrated into the system. YOLOv8 performs:

- Bounding box prediction around detected phones
- Confidence scoring
- Real-time inference at 20–30 FPS depending on hardware

This ensures that the system can instantly recognize when a student uses a mobile phone without any delay.

Step 5: Real-Time Face Recognition

For every frame captured by the webcam, the following operations are performed:

1. Detect faces in the frame
2. Generate encodings for each detected face
3. Compare the encodings with stored training embeddings

4. Choose the closest match using Euclidean distance

This enables the system to immediately identify *which* student is present in the frame.

Step 6: Combined Detection and Alert System

When both conditions are true:

- A mobile phone is detected (via YOLO)
- A student's face is recognized (via embeddings)

Then the system triggers the alert mechanism:

- A snapshot of the event is saved locally with timestamps
- An automated email is sent to the instructor
- A cooldown timer prevents duplicate alerts for the same student within a short period

This ensures reliable and evidence-based monitoring without overwhelming the instructor with repeated alerts.

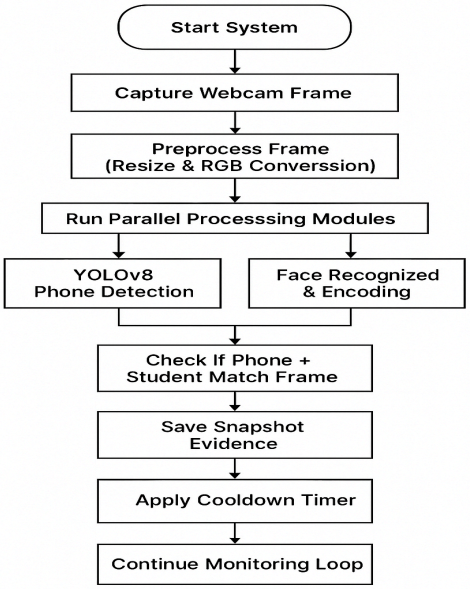


Fig 1. Overall Flowchart of the Proposed System

7. System Architecture

1. Data Acquisition Module

Responsible for capturing:

- Live video frames from the webcam
- Student face images during dataset creation

2. Processing Module

This is the core ML engine that includes:

- YOLOv8 object detection for mobile phone identification
- Face encoding generation and comparison
- Mapping the detected face to a known student

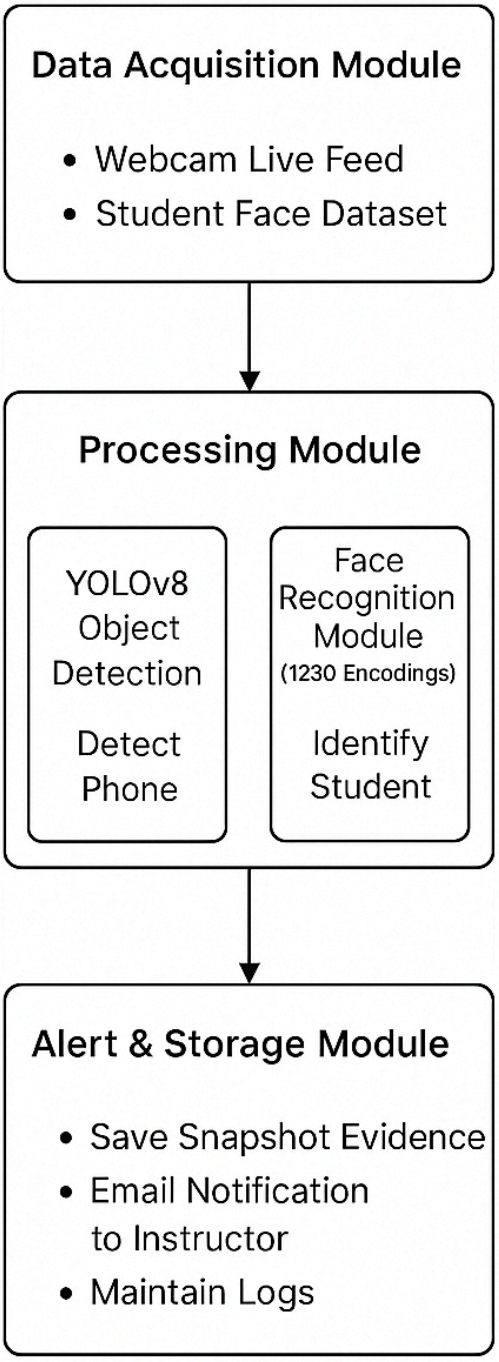
3. Alert & Storage Module

Handles the output actions such as:

- Saving evidence snapshots
- Sending automated email alerts
- Maintaining logs for future analysis

This modular architecture ensures scalability, high accuracy, and easier debugging.

System Architecture



8. Implementation

The implementation of the system is carried out using Python and several machine learning and computer vision libraries. Each module of the project is implemented as a separate script to ensure modularity, ease of debugging, and scalability. The details are as follows:

Programming Language

- Python 3.10
  Chosen for its rich support for computer vision, ML frameworks, and ease of integration.

Libraries Used

- OpenCV – For video capture, image processing, and drawing bounding boxes
- face_recognition – For face detection and 128D embedding generation
- NumPy – For numerical operations and handling encoded vectors
- Ultralytics YOLO – Pretrained YOLOv8 model for mobile phone detection
- SMTP – For sending automated email alerts to the instructor

Files Implemented

1. create_a_dataset.py

Used to capture and store face images for each student to build the training dataset.

2. train_faces.py

Processes collected images, extracts 128D facial embeddings, and saves the encoded results in `.npy` format for fast recognition.

3. main.py

The primary application that handles:

- YOLOv8 phone detection
- Face recognition from webcam frames
- Real-time video stream processing
- Alert generation and email sending

Hardware Requirements

- Standard laptop or PC
- Inbuilt or external webcam for video input
- Minimum 8GB RAM for smooth YOLO + face recognition performance

9. Results & Analysis

The system was tested with multiple students under various lighting conditions, face angles, and distances from the webcam. The performance was evaluated based on phone detection accuracy, face recognition accuracy, frame rate, and the reliability of the alert system.

1. Phone Detection Accuracy

The YOLOv8 model consistently detected mobile phones with high confidence values above 0.70 in most test cases. Even when the phone was partially visible, the system was able to identify it with reasonable accuracy.

2. Face Recognition Accuracy

The face recognition module achieved strong accuracy when the student's face was clearly visible. Recognition quality improved significantly when more training images were used per student. Slight reductions in accuracy were observed in low-light settings or when the face was partially covered.

3. Real-Time System Performance

The system maintained a stable frame rate of 15–20 FPS, providing smooth real-time detection. The end-to-end pipeline (YOLO + face recognition) produced minimal delay, ensuring that alerts were generated almost instantly after detection.

4. Alert System Reliability

The automated email alert mechanism worked effectively. Each detected event generated:

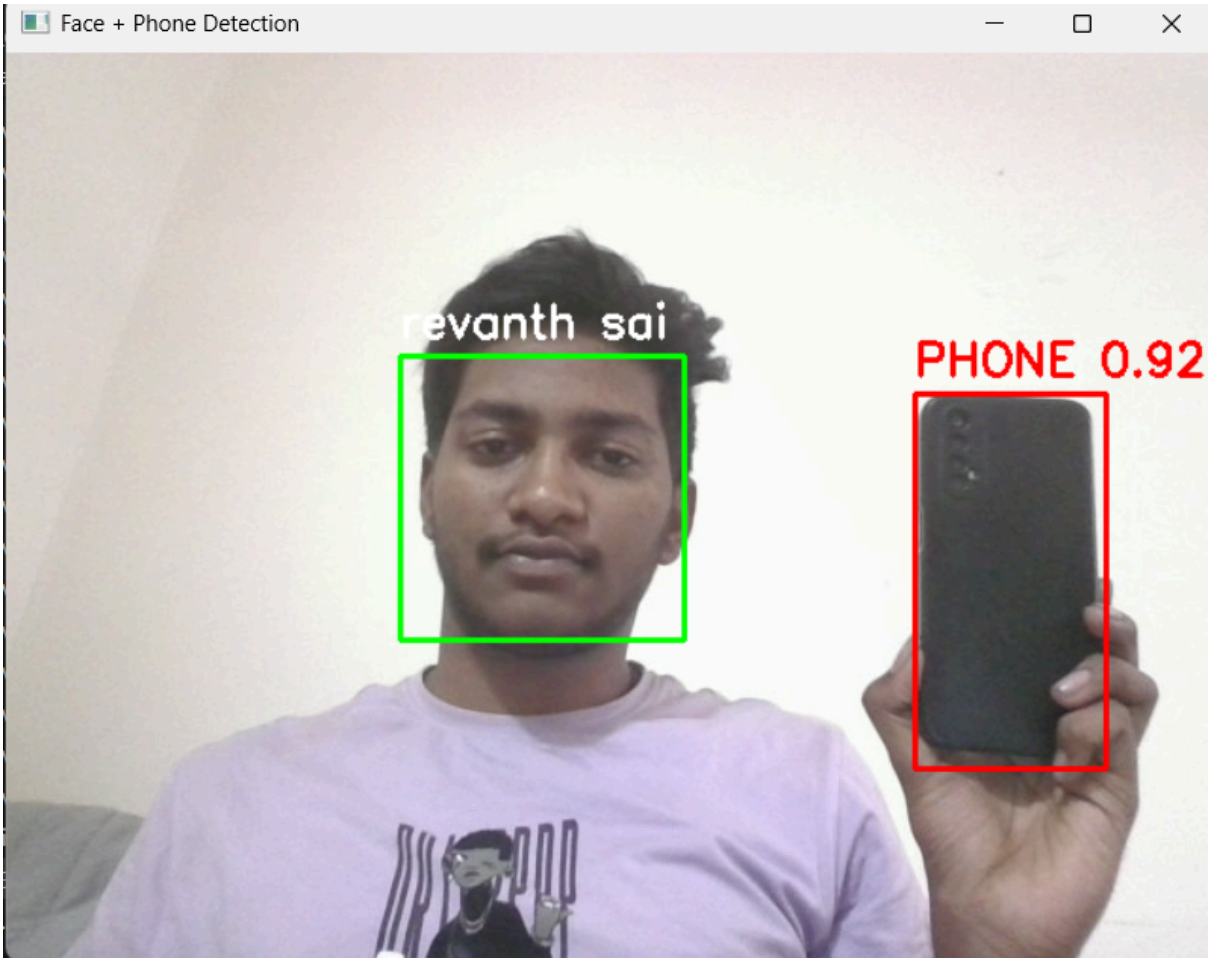
- A timestamped snapshot

- Student identity
- A reliable evidence image sent to the instructor's email

The cooldown timer prevented repeated or spam alerts, improving the usability of the system.

Overall Analysis

The system performed effectively and met all its objectives. It successfully detected prohibited mobile phone usage and accurately identified the student in real time. Despite minor limitations such as lighting sensitivity, the overall results demonstrate that the system is reliable for classroom monitoring.



10. Limitations

1. **Low-Light Sensitivity**
   The system's performance decreases significantly in dim lighting. Both YOLOv8 and the face recognition model rely on clear visual features, which become difficult to detect in poor illumination.

2. **Partial Face Visibility Issues**
   Face recognition may fail when the student's face is partially covered by a hand, mask, or turned away from the camera. This affects the accuracy of identity detection.

3. **Misclassification by YOLOv8**
   YOLO may detect objects similar in shape or size to mobile phones (such as calculators or small notebooks), causing false positives in certain scenarios.

4. **Dependence on Webcam Quality**
   Low-resolution or older webcams reduce image clarity, affecting both phone detection and facial encoding accuracy. A good-quality camera improves performance.

5. **Internet Dependency for Alerts**
   The email alert system requires an active and stable internet connection. If the connection is slow or unavailable, alerts may be delayed or not sent.
6. **Background Disturbances**
   Movement or cluttered backgrounds sometimes confuse detection models, especially when multiple students are simultaneously present

11.Future Scope

1. This project can be extended to:
2. Create a live dashboard
3. Store detection logs in a database
4. Add student attendance tracking
5. Implement head pose + attention monitoring
6. Add push notifications/mobile app alerts
7. Train a custom YOLO model for classroom environments
8. Integrate with Smart Classroom systems

12. Conclusion

This project successfully demonstrates the integration of machine learning, deep learning, and real-time computer vision to solve a practical classroom monitoring problem. YOLOv8 efficiently detects mobile phones, while the face recognition module identifies the student using them.

The system reduces manual monitoring effort, enhances discipline, and provides evidence-based alerts.
Most importantly, the project covers essential ML syllabus topics such as supervised learning, feature extraction, dataset creation, ANN/CNN concepts, model evaluation, and real-time inference.

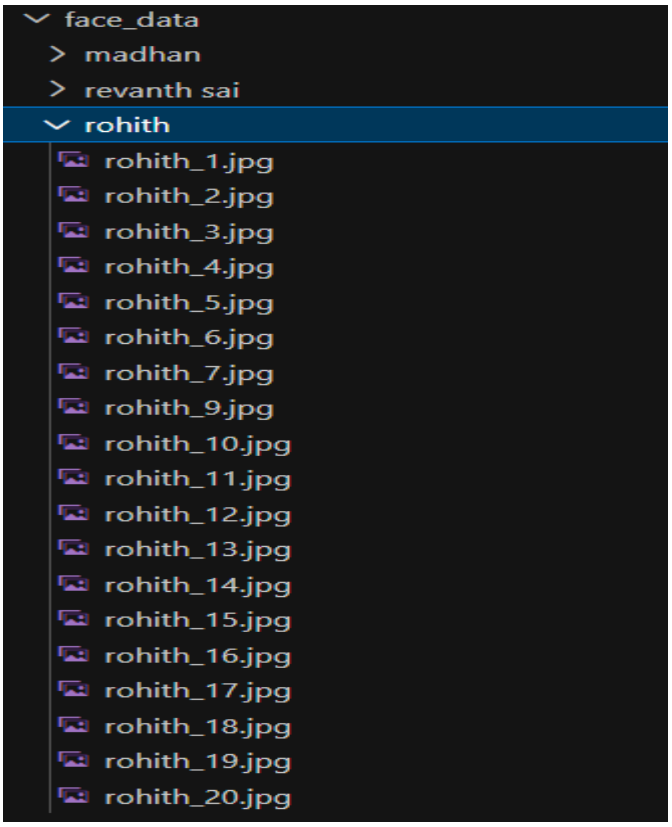
The project is scalable, practical, and highly relevant to modern educational environments.

13.Appendices

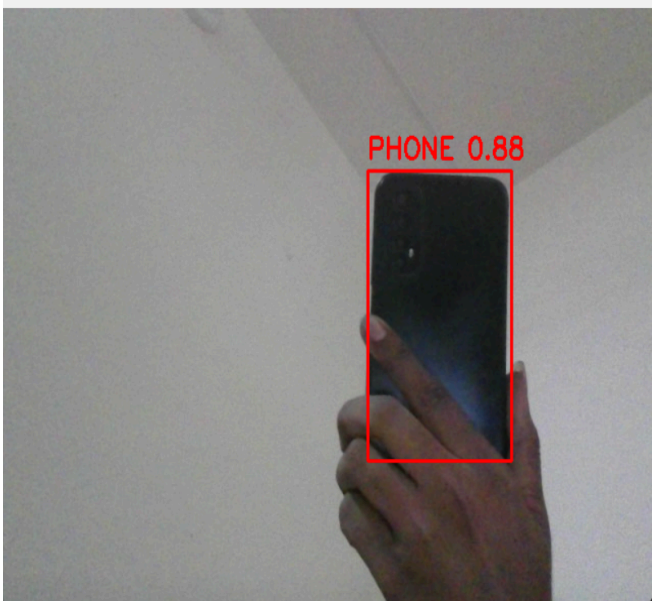
Appendix A – Dataset Images

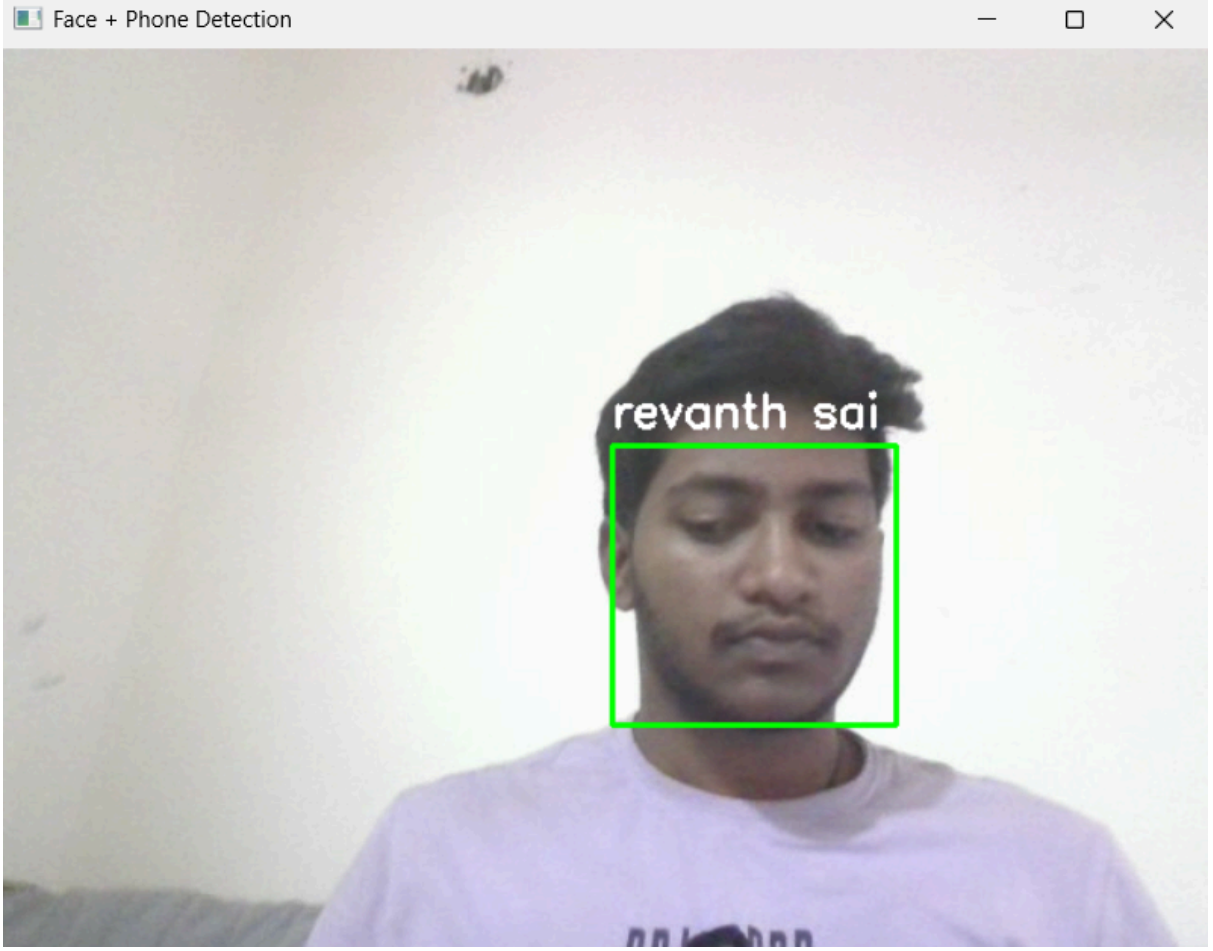
sample1: revanth sai

sample2: Rohith

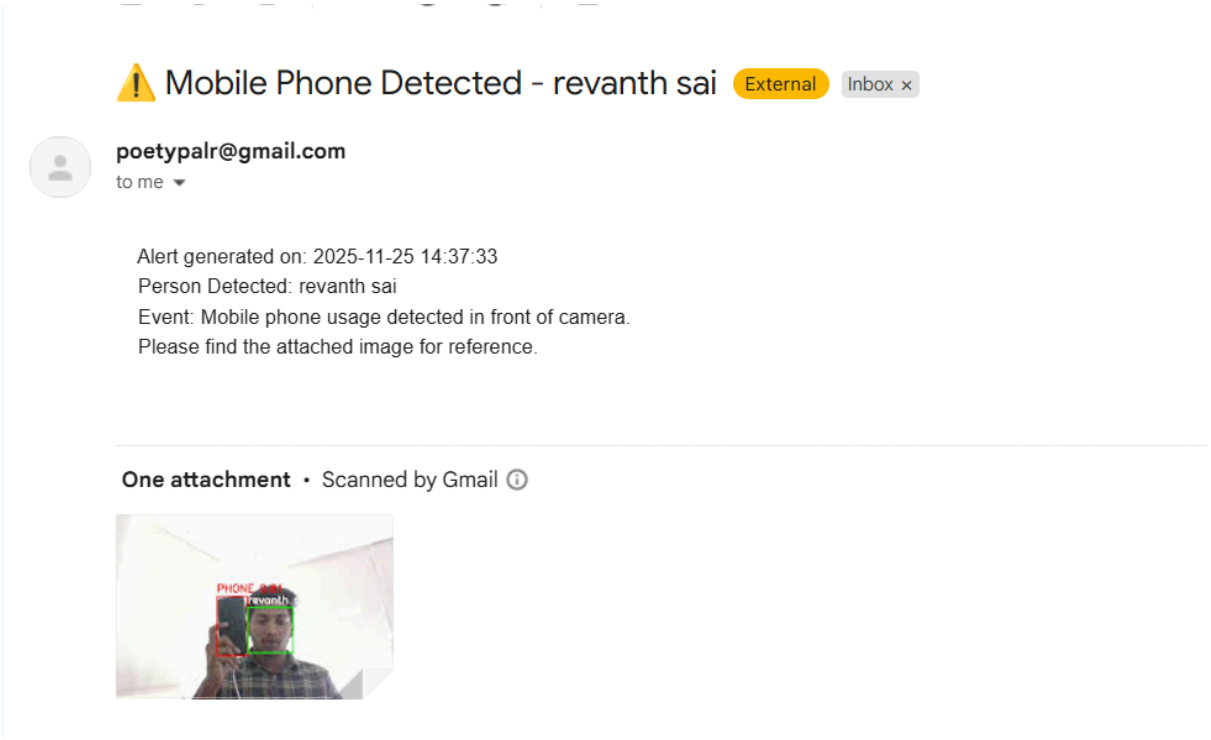


Appendix B – YOLO& Face Detection Screenshot





Appendix C – Email Alert Screenshot



14. REFERENCES

Jocher, G. et al. (2023). *Ultralytics YOLOv8: Real-Time Object Detection Models.*Available at: https://github.com/ultralytics/ultralytics

King, D. E. (2009). *Dlib-ml: A Machine Learning Toolkit.* Journal of Machine Learning Research, 10, 1755–1758.(Face recognition model used for 128-D embeddings)

Hassner, T., Harel, S., Paz, E., & Enbar, R. (2015). *Effective Face Frontalization in Unconstrained Images.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR). (Face alignment concepts)

Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement.* arXiv:1804.02767 (Foundation of YOLO models used in modern variants)

Zidan, M., et al. (2023). *A Survey on YOLO Object Detection Algorithms.* IEEE Access.(Explanation of YOLO architecture & performance)

OpenCV Documentation (2024). *Real-Time Computer Vision Functions.*
Available at: https://docs.opencv.org/

Ageitgey, A. (2023). *Face Recognition Library Documentation.*
Available at: https://github.com/ageitgey/face_recognition (Used for face detection & encoding)

Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning (3rd Edition).* Packt Publishing.(General ML concepts: preprocessing, training, validation)

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd Edition).* O'Reilly Media. (Background on ML workflows & DL fundamentals)

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications.* Springer.
(Foundations of computer vision techniques used)

SMTP Python Documentation (2024). *smtplib — SMTP protocol client.*
Available at: https://docs.python.org/3/library/smtplib.html
(Used in project email alert system)